Reference Notes

Jonathan Ma

July 17, 2026

Contents

Part I
Literature Review

1. CoLD: Counterfactually Guided Length Debiasing for PRMs in Math Reasoning

TLDR: PRMs may rewards verbosity/length rather than correctness (a concrete reward-hacking shortcut). Propose counterfactual length debiasing through penalty, bias estimation, and joint-length invariant training. Empirical and causal motivation

2. PathFinder-PRM: Error-Aware Hierarchical Supervision

TLDR: PRMs need to know what kind of error occured, so the authors propose classifying math/consistency errors first and then combining signals into a correctness reward. Empirical study that improves PRMBench score with less data

3. Towards Hierarchical Multi-Step Reward Models for Enhanced Reasoning in LLMs

TLDR: Stepwise PRMs may mishandle multistep coherence and correction, which makes scores unreliable. The authors propose scoring individual steps and consecutive step blocks, and use hierarchical node compression for data augmentation. An empirical study was conducted and claims more stable best-of-N behavior and cross domain robustness

4. Towards Robust PRMs via Noise-Aware Learning

TLDR: MC labels are policy-dependent and noisy (incorrect steps can have high reards if self correction succeeds). Propose Reflection aware label correction combined with iterative confidence-based denoising. Empirical study reports 27% abosulte F1 gain over noisy-supervision PRMs

5. Uncertainty Calibration of PRMs

TLDR: PRMs are often overconfident and overestimate probability of success. Quantile-Regression calibration and confidence bounds are used for adaptive test time compute, and results are empirical calibration and adaptive scaling

6. Robustness and Hackability of PRMs

TLDR: Directly Studies PRM Hackability; finds that PRMs can reward fluent invalid reasoning and RL can drive near-perfect PRM scores while accuracy stays very low. Diagnostic framework: static perturbations, adversarial optimization, and RL induced hacking. An empirical robustness evaluation that results in a usable tool.

7. Self-Denoising MC Annotation for Robust Process Reward Learning

TLDR: Synthetic MC cupervision is scalable but noisy. Propose selective sampling + self denoising loss to reduce FP/FN. Empirical study with large F1 improvement and lower annotation cost

8. Benchmarking PRMs with Systematic Reasoning Patterns

TLDR: Existing PRM benchmarks miss systematic reasoning-pattern failures. The authors propose to benchmark errors across decomposition and then combine signals into a correctness reward, results improve PRMBench score with less data

9. Lets Verify Step by Step

TLDR: Foundational PRM/process-supervision paper for background. Uses human step-level labels and compares process vs outcome supervision. Empirical study and releases a dataset with 800k labels for training

10. Min-Form Credit Assignment is all PRMs need for Reasoning

TLDR: PRM reward hacking during Rl comes from summing future-step rewards, letting models exploit a few high-reward steps. Propose replacing cumulative rewards with minimum future reward so that one bad step limits trajectory value. Empirical study and claims training collapse under sum-form and improved reasoning with min-form.

Part II
Notes

basics of RL, how it works, and why rewards hackng occurs, is an issue, set up RL experiments. LLMs for math reasoning, benchmarking LLMs, MDPs

11. Reinforcement Learning Overview

RL is a sequential decision-making framework in which agents learn to perform actions in an environment with the goal of maximizing rewards. An RL algorithm might control the moves (*action*) of a character (*agent*) in a video game (*environment*) aiming to maximize the score (*reward*). In Finance, this could be a virtual trader who buys/sells assets on a financial exchange to maximize profit RL is a framework that doesn't necessarily need deep learning but modern systems often use deep networks by encoding the environment and mapping to the next action

Challenges of RL

Consider playing a game of chess. The reward set is $[-1, 0, +1]$ at the end of a game, representing wins, losses, and draws.

- The reward is *sparse* since we must play an entire game to recieve feedback.
- The reward is temporally offset from the action that caused it (an advantage might be gained 30 moves before victory) so we must associate this reward to its critical action. This is known as the *temporal credit assignment problem*
- The environment is stochastic since the opponent doesn't necessarily use the same moves.
- The agent must balance exploring the environment (trying new opening moves) and exploiting what it already knows. This is called *explroation-exploitation trade-off.*

12. Markov Decision Processes (MDPs), Returns, and Policies

RL learns a *policy* that maximizes *expected return* in a MDP.

A Markov process (MP) consists of a set of states and transition probabilities $\mathbb{P}(s_{t+1}|s_t)$ that define the probability s_t moves to s_{t+1}. Being Markovian means that the current state only depends on the previous state.

A Markov reward process (MRP) extends a MP to include a distribution $\mathbb{P}(r_{t+1}|s_t)$ over the possible rewards recieved at the next time step, producing a sequence of $s_1, r_2, s_2, r_3, s_3, r_4, ...$ and includes a discount factor $\gamma \in (0, 1]$ that is used to compute return G_t:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

The return is the sum of the cumulative discounted future rewards, and $\gamma \leq 1$ makes rewards that are closer in time more valuable than rewards further away.

A Markov Decision Process (MDP) adds a set of possible actions at each time step, so the reward depends on action and state:

$$\mathbb{P}(s_{t+1}|s_t, a_t) \quad \mathbb{P}(r_{t+1}|s_t, a_t)$$

which produce tuples $(s_1, a_1, r_2), (s_2, a_2, r_3), (s_3, a_3, r_4), ...$

Partially Observable MPs (POMDP)

In this case, the agent does not have access to the entire space (think a chess piece only being able to move based on everything in a two tile radius). The agent recieves an observation $o_t \sim \mathbb{P}(o_t|s_t)$, generating $s_1, o_1, a_1, r_2, s_2, o_2, a_2, r_3, o_3, a_3, s_3, r_4, ...$

The rules that determine the agent's action for each state are known as a policy, which may be:

- stochastic: policy defines a distribution over actions for each state
- deterministic: agent takes the same action in a given state and zero otherwise
- stationary: depends only on the current state
- nonstationary: depends only on current time step

13. Expected Return

We want to choose a policy that maximizes expected return, so to do so we assign a value to each state and stata-action pair. The return G_t depends on the state s_t and policy $\pi(a|s)$. From this state, the agent will pass through a sequence of states, taking actions and recieving rewards. This sequence differs every time th eagent starts in the same place since the policy, state transitions, and rewards are all stochastic. We characterize how good a state is by considering the expected return:

$$v(s_t|\pi) = \mathbb{E}[G_t|s_t, \pi]$$

which informally tells us the long term reward we can expect. Similarity the state-action value function can tell us the expected reward if we start in a state, take an action, and follow the policy. This is how RL connects future rewards to current actions (thereby resolving the temporal credit assignment problem)

$$q(s_t, a_t|\pi) = \mathbb{E}[G_t|s_t, a_t, \pi]$$

13.1. Optimal Policy

We want a policy that maximizes expected return. For MDPs (but not POMDPs) there is always a deterministic stationary policy that maximisez. If we know this optimal policy, then the optimal state-value function $v^*[s_t]$ and stata-action value function $q^*[s_t, a_t]$ are:

$$v^*[s_t] = \max_\pi[\mathbb{E}[G_t|s_t, \pi]] \text{ and } q^*[s_t, a_t] = \max_\pi[\mathbb{E}[G_t|s_t, a_t, \pi]]$$

so if we knew the optimal action values, then we can derive the optimal policy by choosing the action with the highest value. Some RL algorithms are based on alternately estimating the action values and policy

13.2. Bellman Equations

We may not know the state values or action values for any policy, but we know they must be consistent with one another. The state value can be found by taking a weighted sum of the action values with weights dependent on the PDF of the policy. Similarily, the value of an action is the immediate reward generated by taking the action plus the value $v[s_{t+1}]$ of being in the subsequent state s_{t+1} discouted by γ. This isnt deterministic, so we weight the values according to the transition probabilities:

$$q[s_t, a_t] = r[s_t, a_t] + \gamma \sum_{s_{t+1}} \mathbb{P}(s_t + 1|s_t, a_t)v[s_{t+1}]$$

14. Tabular Reinforcement Learning

Tabilar RL algorithms (ones that don't rely on function approximation) are divided into *model-based* and *model-free mmethods*. Model-based methods use the MDP structure and find the best policy from the transition matrix and reward structure. If these are known, they can be tackled by dynamic programming, else they are estimated from observed trajectories.

Model free methods assume the transition matrix and reward structure of the underlying MDP are unknown, falling into two families:

- Value Estimation: estimate the optimal state-action value function and the assign policy according to the action in each state with the greatest value
- Policy Estimation: estimate the optimal policy using gradient descent without intermediate steps

Within each family, Monte Carlo is used to simulate and TD (temporal difference) updates the policy as the agent traverses the MDP

14.1. Dynamic Programming

Imagine a grid with a penguin looking for a fish, but the grid has holes as well. A DP algorithm follows:

1. Initialization: The state values are initialized at zero, and policy is chosen randomly.
2. Policy Evaluation: The state values are updated to be consistent with their neighbors.
3. Policy Improvement: The policy is updated to mobe the agent to states with the highest value
4. After several iterations, the algorithm converges to the optimal policy

Policy Evaluation We sweep through states s_t an update their values using

$$v[s_t] \leftarrow \sum_{a_t} \pi[a_t|s_t](r[s_t, a_t] + \gamma \sum_{s_{t+1}} \mathbb{P}(s_{t+1}|s_t, a_t)v[s_{t+1}])$$

Each value is consisgtent with the successor state using the Bellman equation for state values, also known as *bootstrapping*.

Policy Improvement To update the policy, we greedily choose the action that maximizes the value for each state: